Knowledge Graph Repair with Reinforcement Learning and Two Rule Prompts

Anonymous Authors

***Abstract*—Knowledge graph repair has two linked tasks: get useful rules and fix graph errors within a budget. We train a Double-DQN policy to choose between them. The test has four graph actions and four rule actions in a fixed rule library. Two text prompts also generate rules. One removes short text spans; the other adds new sentences. We test the generated rules in saved-output repair runs. An action test shows that a penalty based on error counts blocks useful relation repair. The repair can create temporary duplicate edges. We compare count, rate, and zero penalties using forty neural models and ten one-step fitted baselines. On thirty new corruptions of the same base graph, the rate penalty raises Double-DQN triple F1 from 94.67% to 99.49%. It restores 94.00% of the original invalid relations. A simple acquire-then-deficit rule reaches 99.91% F1. The 9,456 saved model responses contain different candidates from the two prompts. With equal call budgets, augmentation yields more constraints. We compile 18,143 exact type patterns. Their union detects 10 of 64 defects in RuleTest-94. The repair loop removes these ten records and keeps all thirty clean records. Further document tests count recovered facts and correct facts lost. The results show how reward design changes repair and how rule coverage shapes the choice of later actions.**

I. Introduction

A. Task and Motivation

A knowledge graph (KG) stores facts as nodes and edges [1]. Graph repair checks and improves these facts [2]. It has two linked decisions: which errors to fix and which rules to get. A repair needs a rule that can detect its target error. Getting a rule has a cost. Its value depends on the errors that remain. A scheduler uses the current graph and rules to choose the next action.

Experts supply rules for known needs. Source text can supply further rules about entity types, hierarchies, and required steps. Large language models (LLMs) can propose these rules. We check the rule format, vocabulary, and matched records before using a proposal to detect errors.

We use reinforcement learning (RL) to divide a fixed budget between graph and rule actions. We also use two prompts to generate rules from text. An active rule changes which repairs are available. A repair changes which rules the graph needs next.

We test this link in three stages. First, a fixed rule library supports comparisons of learned policies, simple schedules, and rewards. Second, saved paired responses show what deletion and augmentation each generate. Third, direct rule execution and document tests show which candidates lead to repairs. The first stage trains policies. The third stage uses fixed schedules.

B. Contributions

We make three contributions:

1) **A graph–rule control environment.** Fourteen input features describe the graph, rules, and budget. Eight actions change graph records or active rules. An action mask lists the allowed choices. The reward includes quality gains and action costs.
2) **A Double-DQN policy and shared-condition tests.** Temporary duplicates explain why the count penalty blocks relation repair. Forty neural models and ten one-step fitted baselines compare three penalties. Component tests measure graph features, rule features, action masking, and call cost. The rate penalty improves repair, and the action mask raises final F1.
3) **Two rule-generation prompts and source-linked tests.** Deletion and augmentation share an output format. Equal-call tests measure candidate counts. Execution logs link each type pattern to its original response. Fixed schedules and document tests measure repairs, correct facts kept, and the effect of rule order.

Figure 1 shows the method. The policy receives fourteen features for graph quality, rule quality, remaining errors, remaining budget, and active rule shares. The action mask lists allowed actions. After each action, the environment updates quality and reward. Separate action tests inspect graph changes from saved states.

C. Paper Outline

Section II reviews earlier work. Section III gives the scheduling model and rule prompts. Section IV gives the three test stages and discussion. The appendices give protocols, execution logs, and further repair tests.

II. Related Work

A. Graph Quality and Repair

Graph quality includes correct facts, missing facts, and agreement with rules [3]. Test-based checks find violations of data requirements [4]. Statistical methods use value distributions to find likely errors [5]. Graph refinement also covers adding missing facts and removing wrong ones [2]. Our scheduler uses quality scores to choose graph edits and rule actions under one budget.

B. Reinforcement Learning for Knowledge Graphs

MINERVA by Das et al. [6] learns paths for multi-hop queries. DeepPath by Xiong et al. [7] uses RL to find reasoning

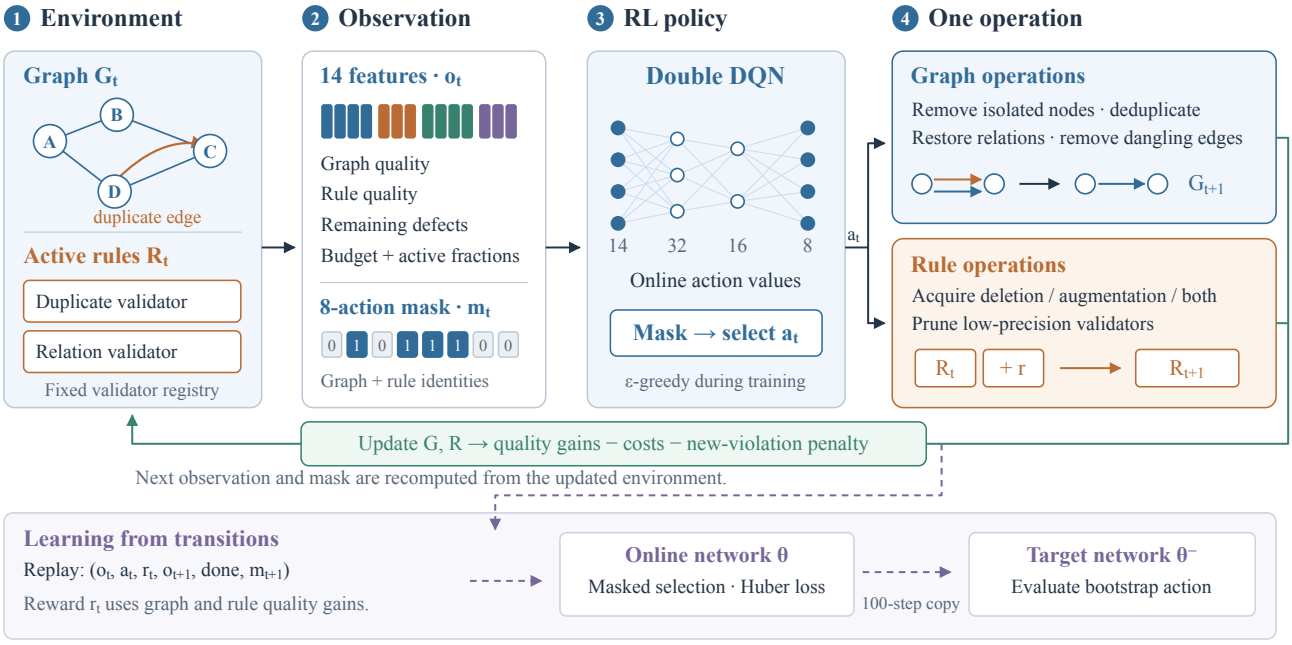


Fig. 1. Learning graph and rule actions. The environment gives fourteen inputs and a mask of eight actions. The policy chooses one action. The environment updates quality and reward. The lower band shows Double-DQN replay training. Small graphs and mask entries show example states. The loop uses a fixed rule library. Figure 2 gives the text prompts.

paths. Lin et al. [8] study rewards for multi-hop reasoning. These methods choose paths to answer queries. Our actions change graph records and activate rules for later repairs.

We compare learned policies with fixed schedules and simple rules that use the current input. All use the same graph operations. Retrained component tests measure input features and action masking. The fixed rule library allows each graph change to be replayed. Further tests use rules from text.

C. Language Models for Knowledge Engineering

KG-BERT [9] uses a language model to classify triples. KEPLER [10] joins language modeling with knowledge embeddings. Pan et al. [11] review links between language models and KGs. These studies use text to help build and complete graphs.

Chain-of-thought prompts [12] and ReAct [13] use reasoning steps and tools. Our two prompts share a JSON format and use one call each. Deletion asks for rules from removed text spans. Augmentation asks for rules from added sentences. We save the source of each candidate and test the patterns that compile.

D. Constraint Languages, Rule Mining, and Repair

SHACL [14] and ShEx [15] express graph constraints. The supplied shapes decide which errors they check.

AMIE [16] mines Horn rules from graph facts. RuDiK [17] finds rules in knowledge bases. Neural LP [18] learns logical rules through differentiable operations. pLogicNet combines logical rules with graph embeddings [19]. RNNLogic trains a rule generator and an answer predictor together [20]. Our candidates come from documents. They include type patterns and statements about required steps. A compiler turns supported patterns into checks, then execution tests measure the matched records.

Lin et al. [21] test LLM-based graph repair. We study the order of rule acquisition and graph edits. Source logs and execution records show how generated candidates reach the repair loop.

III. Method

A. Overview

The method stores a graph G_t and rules R_t. At each step, a policy gets rules, removes rules, or repairs the graph with an active rule. New rules can allow later repairs. Remaining errors guide the choice of further rules. A fixed budget limits the actions.

Figure 1 shows the scheduling loop. Figure 2 shows the two rule prompts. The scheduling test uses a fixed rule library. Further tests compile saved candidates and run them through a repair interface. Document tests get one response at a time and check source rules.

B. Quality Scores

1) Graph Quality: After each graph action, code counts isolated nodes, repeated edges, invalid relations, and edges with missing endpoints. Their rates are I_r, D_r, V_r, L_r. The four quality scores are

$$S(C_1) = 1 - I_r, \quad S(C_2) = 1 - D_r,$$
$$S(C_3) = 1 - V_r, \quad S(C_4) = 1 - L_r. \tag{1}$$

The test uses

$$Q(G) = 0.20S(C_1) + 0.20S(C_2) + 0.30S(C_3) + 0.30S(C_4). \tag{2}$$

These fixed weights define the structural quality score.

2) Rule Quality: We score active rules on a fixed set of 180 calibration cases. This set is stored apart from the graph corruption list. The union of rule predictions gives

$$\text{Precision}(R) = \frac{TP}{TP + FP}, \qquad \text{Recall}(R) = \frac{TP}{TP + FN}. \tag{3}$$

An empty rule set has zero precision and recall. Coverage is the share of four error families with an active rule. The online score is

$$\begin{aligned} Q_{\text{online}}(R) = {} & 0.35\,\text{Precision}(R) + 0.35\,\text{Recall}(R) \\ & + 0.30\,\text{Coverage}(R). \end{aligned} \tag{4}$$

Each high-precision module detects 27 of 30 family-positive cases and two clean cases in the calibration set. Three more modules have lower precision. Rule scores use this fixed set. Generation tests count candidates from saved responses.

C. Learning to Choose Graph and Rule Actions

1) Environment and Policy Inputs: The state stores the graph, active rules, budget, initial error counts, and trusted relation-restoration records. The environment holds these restoration records. The policy receives fourteen values:

$$\begin{aligned} o_t = [& S(C_1), S(C_2), S(C_3), S(C_4), \\ & \text{Precision}, \text{Recall}, \text{Coverage}, \tilde{n}_{\text{iso}}, \tilde{n}_{\text{dup}}, \\ & \tilde{n}_{\text{invalid}}, \tilde{n}_{\text{dang}}, \tilde{b}, \rho_{\text{valid}}, \rho_{\text{noisy}}]_t. \end{aligned} \tag{5}$$

We divide each error count by its episode-initial count, using at least one as the denominator. The last entries give the remaining budget share and active shares of high- and low-precision modules. Each input value is clipped to $[0, 1.5]$. Coverage equals ρ_{valid} in this library. Both inputs remain in the shared network design. The network uses these fourteen values and the action mask.

The eight actions are

$$\begin{aligned} \mathcal{A} = \{ & a_{\text{iso}}, a_{\text{dup}}, a_{\text{rel}}, a_{\text{dang}}, \\ & a_{\text{del}}, a_{\text{aug}}, a_{\text{mix}}, a_{\text{prune}}\}. \end{aligned} \tag{6}$$

The first four actions remove isolated noise nodes, remove duplicate edges, restore invalid relation names, and remove edges with missing endpoints. Each handles a batch based on 60% of the remaining matching errors. Relation repair uses the private corruption record after detection.

The other actions activate a deletion module, activate an augmentation module, activate one module from each family, or remove modules with calibration precision below 0.5. Acquisition follows the fixed library order. Repair requires an active rule and a matching error. These actions activate stored modules. The text prompts below supply candidates for further execution tests.

The graph and active rule IDs determine the action mask. All compared policies receive this mask, except the no-mask setting. Policies receive the current inputs and mask. A separate model-informed baseline can copy the environment and try future actions.

2) Reward and Double DQN: The reward uses $\Delta Q(G_t) = Q(G_{t+1}) - Q(G_t)$ and the change in rule score. It subtracts costs for acquisition, edits, and new errors:

$$\begin{aligned} r_t = {} & \Delta Q(G_t) + 0.4\Delta Q_{\text{online}}(R_t) \\ & - 0.004C_{\text{call}} - 2 \times 10^{-5} C_{\text{edit}} - P_t. \end{aligned} \tag{7}$$

C_{call} counts one unit for single-family acquisition and two for mixed acquisition. These units measure simulated call cost. Pruning has zero call cost. The environment computes N_{new} from error IDs before and after the action. Each node and edge copy has its own fixed ID. The edit count includes graph changes and active-rule changes. Logs also give these two counts apart.

An episode ends when $Q(G) \geq 0.999$ and $Q_{\text{online}}(R) \geq 0.90$, after 18 decisions, or when no action is allowed. An action that would empty a nonempty node or edge set is undone and ends the episode.

Let $\mathcal{V}(G)$ be the multiset of detected error IDs. Then

$$N_{\text{new}} = \left| \mathcal{V}(G_{t+1}) \setminus_m \mathcal{V}(G_t) \right|, \tag{8}$$

Here $\setminus_m$ is multiset difference. Removing an edge with a missing endpoint can leave its other endpoint isolated. One old error disappears and a new one appears. The total count can stay the same. The new term charges for the new error. Each step saves its new error IDs and reward parts.

We compare three fixed choices of P_t. Let $n_{k,t}$ count new error IDs in family k. Let w_k be the weight in Equation 2. The penalties are

$$\begin{aligned} P_t^{\text{count}} &= 0.01 \sum_k n_{k,t}, \\ P_t^{\text{rate}} &= \sum_k w_k \frac{n_{k,t}}{D_{k,0}}, \qquad P_t^{\text{zero}} = 0, \\ D_{k,0} &= \begin{cases} \max(1, |V_0|), & k = \text{isolation}, \\ \max(1, |E_0|), & \text{otherwise}. \end{cases} \end{aligned} \tag{9}$$

V_0 and E_0 are the nodes and edge copies after initial corruption. The denominators stay fixed during the episode. The rate penalty divides new error counts by the initial graph size. Call and edit costs keep their fixed weights. All three settings share inputs, masks, and graph operations. A duplicate edge can repeat a correct fact. We score structural errors and wrong facts apart.

For $J_t = Q(G_t) + 0.4Q_{\text{online}}(R_t)$, discounted quality gains satisfy

$$\begin{aligned} \sum_{t=0}^{T-1} \gamma^t (J_{t+1} - J_t) = {} & -J_0 + \gamma^{T-1} J_T \\ & + (1-\gamma) \sum_{t=1}^{T-1} \gamma^{t-1} J_t. \end{aligned} \tag{10}$$

Algorithm 1 Deletion-completion steps

Require: Document ID d, source text T
1: $(T^-, F) \leftarrow \text{DeleteShortSpans}(T)$
2: $P \leftarrow \text{DeletionPrompt}(T, T^-, F)$
3: $Y \leftarrow \text{LLM}(P)$ ▷ One call
4: Parse the JSON candidate categories in Y
5: Log $(d, \text{deletion}, F, Y)$
6: **return** grouped candidates, including parse status

The policy thus values early gains and final quality.

A deep Q-network (DQN) predicts the value of each action. We use DQN [22] with the Double-DQN target [23]. The online Q-network has layers $14 \to 32 \to 16 \to 8$ and ReLU activations. The replay buffer stores $(o_t, a_t, r_t, o_{t+1}, d_t)$ and the next-action mask. For a nonempty next-action set, a^* chooses the highest online-network value. The target is

$$\begin{aligned} a^* &= \arg\max_{a' \in \mathcal{A}_{t+1}} Q(o_{t+1}, a'; \theta), \\ y_t &= \begin{cases} r_t, & d_t = 1 \text{ or } \mathcal{A}_{t+1} = \emptyset, \\ r_t + \gamma Q(o_{t+1}, a^*; \theta^-), & \text{otherwise}, \end{cases} \\ \mathcal{L}(\theta) &= \mathbb{E}[\text{Huber}(Q(o_t, a_t; \theta) - y_t)]. \end{aligned} \tag{11}$$

The bootstrap term is zero at a terminal state or an empty allowed set. We copy target parameters every 100 environment steps. Standard DQN uses $\max_{a'} Q(o_{t+1}, a'; \theta^-)$ as its next-state target.

We use Adam [24] with learning rate 0.002. The discount is 0.95. Replay capacity is 20,000, batch size is 64, and warm-up is 128 transitions. Each model trains for 250 episodes. Exploration falls linearly over 260 episodes. The last training episode uses $\epsilon \approx 0.0902$; the lower limit is 0.05. The reward study saves ten checkpoints per setting: Double DQN with count, rate, or zero penalty, and DQN with rate penalty. Appendix F gives the earlier sixty-model count-penalty study. Training runs in the local environment with zero API calls.

D. Generating Rules with Two Prompts

The two prompts propose rules from source text. Each uses one call per document and strategy. The JSON response contains entity types, relation types, allowed and forbidden type patterns, hierarchy statements, geographical hierarchy patterns, and statements about required steps. Equal-budget tests count type declarations and constraints apart.

1) Deletion Completion: The code replaces short text spans with missing-span markers. By default, it chooses up to five start points and at most six characters per span. The prompt includes the original text, changed text, removed spans, and shared JSON format. It asks for missing rule information and reusable constraints. The model compares both versions.

2) Augmentation Expansion: The augmentation prompt includes the source and asks for up to three reasonable extra sentences. The same response gives rules from these sentences. The sentences guide the search for further candidates.

Algorithm 2 Augmentation-expansion steps

Require: Document ID d, source text T
1: $P \leftarrow \text{AugmentationPrompt}(T)$
2: $Y \leftarrow \text{LLM}(P)$ ▷ Sentences and candidates in one call
3: Parse the JSON candidate categories in Y
4: Log $(d, \text{augmentation}, Y)$
5: **return** grouped candidates, including parse status

3) Compiling and Checking Rules: We save each candidate's document, strategy, response line, category, and position. The compiler supports exact subject-type, relation, and object-type triples. A forbidden pattern $r = (\tau_s, p, \tau_o)$ flags a record x when

$$V_r(x) = \mathbf{1}[(\text{type}(s_x), p_x, \text{type}(o_x)) = r]. \tag{12}$$

An allowed pattern permits that exact combination. A record is flagged by a matching forbidden rule. If both kinds match, the checker records a conflict and skips the decision. It trims spaces at the ends of strings and matches the remaining text exactly.

The log keeps malformed candidates, unsupported families, unmapped terms, and patterns with zero matches. RuleTest-94 supplies stored expected outcomes for execution tests. Hand-written checks for required steps and structural errors form a separate comparison.

4) Removing Copies and Saving Sources: The saved code removes identical candidates within each category using a fixed text format. It counts entity and relation declarations apart from constraints.

The generation script defaults to Qwen3-32B [25] and temperature 0.2. Historical call records store parsed outputs and document information. Appendix A gives the prompt builders, translations, and recorded settings. The supplement supplies the literal templates. The separate extraction test uses Qwen3.8-27B-no-thinking.

E. Repair with Saved Generated Rules

The repair interface compiles saved allowed and forbidden type patterns with the same exact-match rule. Activation updates the error report. A forbidden rule with no matching allowed rule allows record removal. After removal, the interface checks matches and allowed actions again. Each match saves its rule ID and original source. Unsupported candidates are logged. Conflicting patterns cause the checker to skip the decision.

The replay adapter provides reset, observation, available-action, and step functions. Its four commands load deletion rules, load augmentation rules, remove forbidden matches, or stop. The policy sees record, flag, conflict, and active-rule counts, acquisition status, budget, and mask. Each bank loads once. Stop or six decisions end replay. This adapter runs fixed schedules. The eight-action environment supplies training rewards.

Missing endpoints or unknown types cause the checker to skip a record. A allowed rule arriving after removal is logged against that removed record. A removal that would empty a

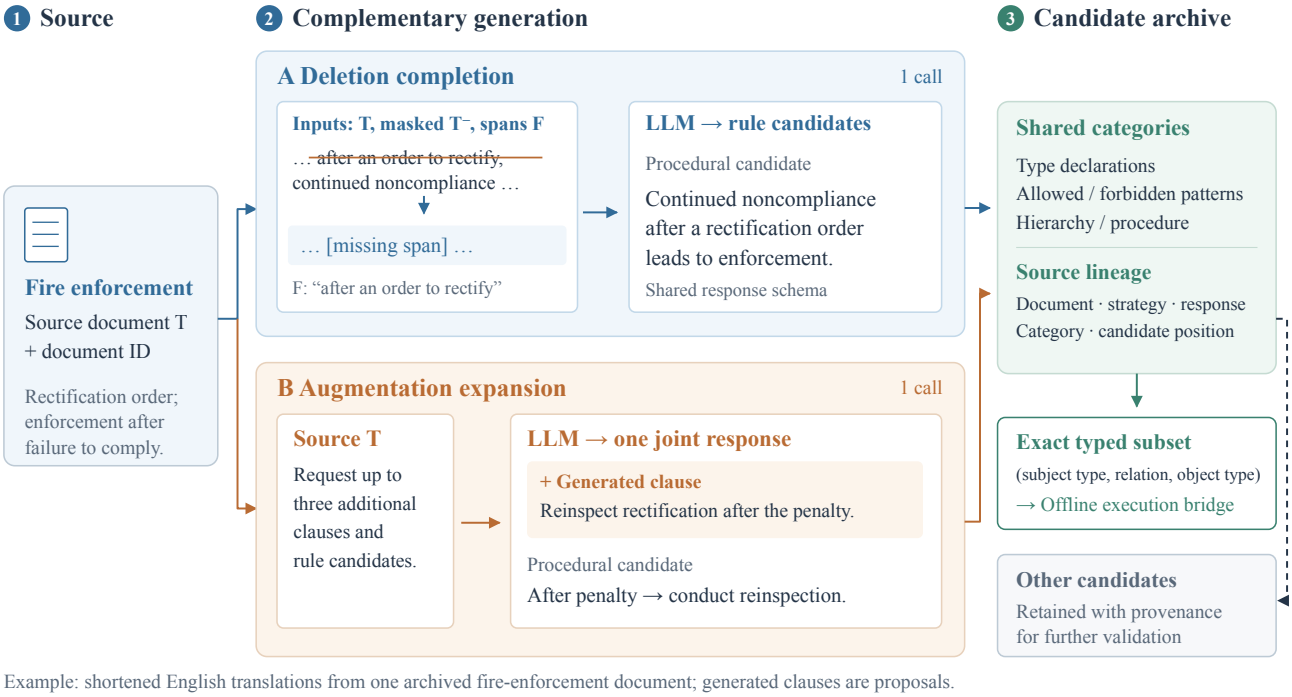


Fig. 2. Two rule prompts, with short English examples from a saved fire-enforcement document. Deletion receives original text, changed text, and removed spans. Augmentation returns extra sentences and candidates in one response. Each candidate keeps its source record. Exact type patterns enter the repair interface.

nonempty set is blocked. Logs save activations, detections, removals, and their sources. The scorer reads reference labels. Removing a record leaves it absent. Loading a full saved bank has its own acquisition count.

a) Document development environment.: The document environment gets one saved response at a time. Its four actions acquire deletion, acquire augmentation, repair, and stop. An episode allows four responses and ten decisions. The reward counts errors found by new rules apart from errors created by edits (Appendix C-A). Fixed schedules and all-path replay measure repair coverage. Eight-action checkpoints run in their original rule library.

b) Rule approval for each document.: The admission layer links each candidate to its response, source documents, initial records, and relation vocabulary. A rule needs valid source links and approval from the chosen review channel. It becomes active when its response is acquired. Permissions, support rules, and forbidden rules share this requirement. Type review covers all initial matching records in the document pair. New or changed records need new review. Model judgments and independent reviews have separate channels.

IV. Experiments

A. Test Design

1) Research Questions: We test the three steps that make a rule useful: acquire it, generate its candidate, and execute it to repair a record. **RQ1:** How does learned scheduling compare with simple policies under fixed rules and graph operations? **RQ2:** Which reward terms and policy parts change repair and correct facts kept? **RQ3:** What do the two prompts add at equal call budgets, and which repairs do their candidates support?

The fixed rule library tests RQ1 and RQ2. Saved generation responses, rule execution, and document tests address RQ3. The document stage uses fixed schedules and measures candidate coverage.

2) Data and Test Rules: The policy environment uses 450 balanced TNEWS/CLUE documents [26], 30 per category. The graph has 873 nodes and 1,083 relations. Independent seeds add isolated nodes, duplicate edges, invalid relations, and edges with missing endpoints. The reward study has twenty development scenarios and thirty test scenarios from the same base graph. The earlier ten scenarios supply action tests and historical component results. A separate 180-case set supplies online rule precision and recall. Acquisition activates stored modules. After detection, relation repair reads the environment's private restoration record.

The rule study uses 9,456 successful saved responses: deletion and augmentation for each of 4,728 government-service documents. RuleTest-94 has 30 clean and 64 defective records with stored expected labels. DocRED [27] supplies entity types and source text for document tests (Appendix C). Appendix H gives further structural and extraction tests.

3) Replay Records: Before training, we fix sources, seeds, costs, and episode budgets. A second manifest records model and scorer hashes before the test run. We retain all predefined policies and failed outcomes. Policies receive current observations and masks. The scorer uses reference labels. Appendix E gives splits and inference rules. Appendix G lists saved steps and replay commands. Local training and saved-output replay

use zero API requests. Generation costs have separate records.

B. Learning Graph and Rule Actions

1) Checking the Reward: The earlier count-penalty Double DQN introduces zero violations. Its ten saved action logs also contain zero relation repairs and leave 61.3 invalid relations per graph. We try each allowed action from saved states. Twenty Double-DQN and heuristic action logs give 2,214 one-step trials, including 311 relation-repair trials. These trials run after the policy evaluation.

All 155 allowed relation-repair trials from Double-DQN states have negative immediate reward with the count penalty. Across these overlapping trials, repairs create 443 duplicate edges. They add zero wrong reference facts and remove zero correct facts. Mean immediate reward changes from -0.01990 with the count penalty to $+0.00821$ with the rate penalty. Repair followed by duplicate removal gives two-step discounted returns of -0.01781 and $+0.01030$. The count penalty blocks useful repair by charging for temporary duplicates.

2) Policies and Scores: We train Double DQN with each of the three penalties and DQN with the rate penalty. All eight policies use the same fourteen features, action mask, and eighteen-step limit. The *acquire-then-deficit* heuristic gets missing high-precision modules in mixed, deletion, then augmentation order. It then chooses the allowed graph action with the largest weighted quality deficit. Random-valid and rule-first-valid give two further schedules.

The one-step ridge baseline fits quality gain, edit count, and new-error rate burden for each action. Each of ten fits uses 250 episodes of uniform allowed-action exploration. Neural models have the same episode budget. Ridge receives the fourteen public features and observed reward parts. Each action has its own training-data standardization. The ridge coefficient is one, with an unpenalized intercept. At inference, predicted edits and burden are clipped at zero. The policy chooses the highest predicted rate reward, including call cost. Ridge uses separate random action logs. Policy inputs contain current features and masks.

Let F^*, F_0 and F_T be unique triple sets for the clean reference, corrupted input, and final graph. The primary reference score is

$$\mathrm{F1}_{\text{triple}} = \frac{2|F_T \cap F^*|}{|F_T| + |F^*|}. \tag{13}$$

The second primary score counts original invalid edge IDs that remain with their reference relation restored. Removed edges receive zero restoration credit, including repaired copies removed during deduplication. Triple F1 counts unique facts. The share of correct facts kept is $|F_T \cap F_0 \cap F^*|/|F_0 \cap F^*|$. The clean extraction and controlled corruptions supply these reference sets.

We also count remaining errors, new errors by family, correct and wrong fact changes, edits, and acquisition units. Final joint quality and quality over the shared budget are

$$\begin{aligned} \bar{Q}_t &= \frac{Q(G_t) + 0.4Q_{\text{online}}(R_t)}{1.4}, \\ \mathrm{AUC}_{18} &= \frac{1}{18}\sum_{t=0}^{17} \frac{\bar{Q}_t + \bar{Q}_{t+1}}{2}. \end{aligned} \tag{14}$$

For an early stop, the final quality is carried forward to step eighteen. All evaluated returns use the rate penalty. The archive also stores returns under count and zero penalties. Acquisition units measure simulated cost. Actual API requests are zero.

3) Policy Results: Count-penalty Double DQN makes zero relation repairs and leaves 60.03 invalid relations per graph (Table I). Its triple F1 is 94.67%. Rate-penalty Double DQN restores 94.00% of original invalid edge copies and reaches 99.49% F1. The F1 gain is 4.824 points (95% CI: $[4.403, 5.141]$). Both primary count–rate tests pass Holm correction ($p = 0.0156$). The rate policy averages 3.92 relation-repair actions and leaves 3.57 invalid relations per graph.

Zero penalty reaches 99.75% F1 and 99.06% restoration. Ridge reaches 99.88% and 97.90%. Acquire-then-deficit reaches 99.91% and 98.77%. These three have higher means than rate Double DQN. The six paired comparisons have Holm-adjusted $p > 0.05$ (Table II). Rate-policy restoration has a standard deviation of 12.44 points across seed means. Figure 4 shows this variation.

All eight policies keep every initially correct reference fact and add zero new wrong reference facts. New violations are duplicate edges. Rate Double DQN adds 4.20 per graph, zero penalty adds 4.42, and the heuristic adds 4.37. Their discounted returns under the shared rate reward are 0.3806, 0.3803, and 0.3877. The heuristic uses 4.00 acquisition units, compared with 4.42 for rate Double DQN.

4) Statistics and Component Tests: We average the thirty scenarios within each training or run seed. The statistical units are ten paired seed means. We use 10,000 paired bootstrap draws and exact two-sided sign randomization over all 2^{10} assignments. Holm correction [28] covers eight planned comparisons: rate Double DQN against count, zero, ridge, and acquire-then-deficit for both primary scores. Each interval measures seed variation on the fixed test set. Fixed schedules repeat the same outcome across seeds.

Appendix F gives the earlier count-penalty component tests. The next tests use the rate reward to measure features, masking, acquisition cost, and penalty size.

5) Component Tests with the Rate Reward: We test five changes: remove graph features, remove rule features, remove the action mask, remove training call cost, or use a small count penalty. Ten seeds and 250 episodes per setting train fifty new models. The ten existing rate-DDQN checkpoints give the full-policy baseline. Thirty further corruptions of the same base graph give 2,100 outcomes, including the heuristic. We evaluate every fixed model.

Feature removal sets the chosen inputs to zero during training and testing. Other inputs and the mask stay available. The no-mask setting chooses from all eight actions during action selection and target updates. Unavailable graph actions can leave the graph unchanged. Repeated acquisition can incur

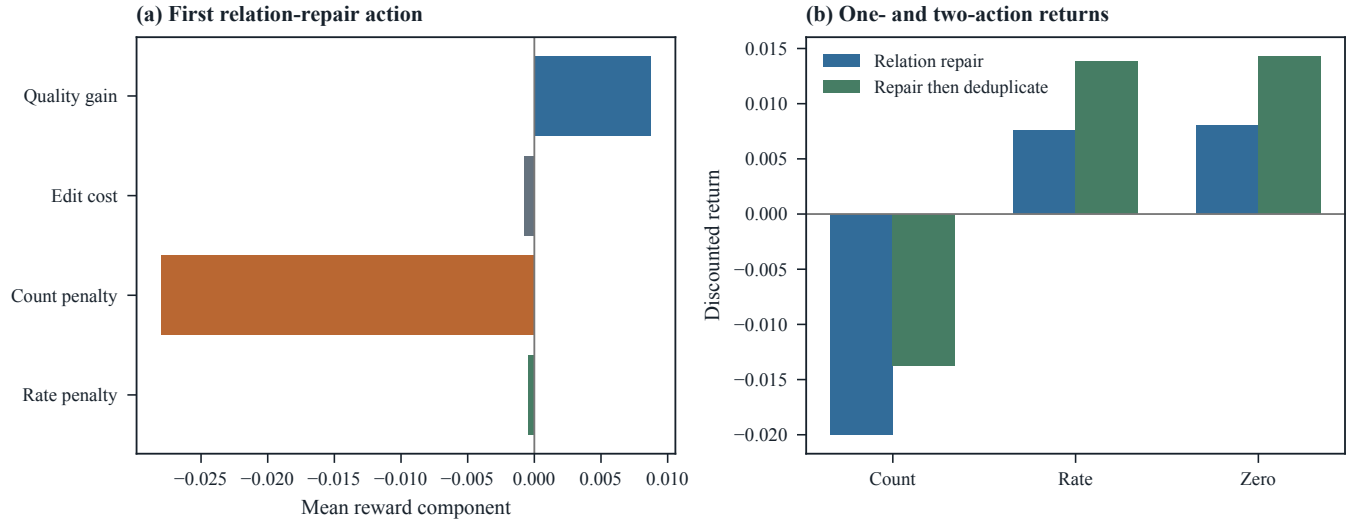


Fig. 3. Reward parts on ten earlier scenarios. Each trial starts from the corrupted input with four high-precision rules active. (a) Mean quality gain, edit cost, and each penalty for the first relation repair. (b) Return for that action and for repair followed by duplicate removal, with discount 0.95. Panel (a) shows the two penalties separately.

TABLE I
Rewards on thirty new corruptions of the same base graph. Means and standard deviations use ten run-seed means. F1 checks unique reference triples. Restoration counts original bad edges kept with their reference relation. All returns use the rate penalty.

Policy	Triple F1 (%)	Restored (%)	Facts kept (%)	New errors	Calls	Return
DDQN (count)	94.67 ± 0.00	0.00 ± 0.00	100.00	0.00	6.27	0.3692
DDQN (rate)	99.49 ± 0.64	94.00 ± 12.44	100.00	4.20	4.42	0.3806
DDQN (zero)	99.75 ± 0.31	99.06 ± 1.37	100.00	4.42	4.06	0.3803
DQN (rate)	99.77 ± 0.34	96.94 ± 6.28	100.00	4.33	4.21	0.3818
One-step ridge	99.88 ± 0.01	97.90 ± 0.15	100.00	4.37	4.04	0.3874
Acquire + deficit	99.91 ± 0.00	98.77 ± 0.00	100.00	4.37	4.00	0.3877
Rule-first valid	99.40 ± 0.00	94.73 ± 0.00	100.00	4.27	8.00	0.3514
Random valid	97.84 ± 0.19	70.72 ± 3.03	100.00	3.12	9.72	0.3394

TABLE II
Rate DDQN minus each baseline in F1 points. Intervals resample ten run-seed means from the same thirty scenarios. Exact sign tests use Holm correction across eight comparisons.

Baseline	Metric	Difference	95% interval	Holm p
DDQN (count)	Triple F1	+4.824	$[+4.403, +5.141]$	0.0156
DDQN (count)	Restoration	+94.000	$[+85.976, +99.919]$	0.0156
DDQN (zero)	Triple F1	-0.255	$[-0.673, +0.094]$	0.9375
DDQN (zero)	Restoration	-5.057	$[-12.825, +0.509]$	0.9375
One-step ridge	Triple F1	-0.387	$[-0.810, -0.069]$	0.1875
One-step ridge	Restoration	-3.898	$[-11.927, +2.006]$	0.9375
Acquire + deficit	Triple F1	-0.415	$[-0.837, -0.099]$	0.1875
Acquire + deficit	Restoration	-4.772	$[-12.796, +1.147]$	0.9375

call cost. The no-call-cost setting removes that training reward term. Evaluation uses the shared rate reward.

Removing the mask lowers F1 by 2.50 points (95% seed-bootstrap CI: $[-3.45, -1.62]$; Holm $p = 0.0234$). It gives 8.26 unavailable choices per episode. This is the one comparison with adjusted $p < 0.05$ among twelve F1 and restoration tests. Removing rule features lowers mean F1 and raises acquisition cost; its adjusted $p > 0.05$. Every setting keeps all initially correct reference facts. The heuristic reaches the highest mean F1, 99.91%.

a) Testing penalty size.: Twenty development rollouts sample allowed actions uniformly. We set the count coefficient to total rate burden divided by total new violations: $\alpha = 0.000164493$. Calibration uses observed graph changes. Small-count DDQN reaches 99.05% F1; rate DDQN reaches 99.52%. The paired difference is -0.47 points (CI: $[-1.73, 0.46]$, Holm $p = 1$). The interval includes zero at the base graph size.

6) Fixed Policies on Larger Graphs: We build one, two, four, and eight disjoint copies of the clean base graph. Each copy has separate node IDs. The largest input has 6,984 nodes and 8,664 edges. At each size, ten policy seeds and five shared corruption seeds compare rate, original-count, and small-count DDQN with acquire-then-deficit. This gives 800 outcomes. All models train at the base size. The test uses repeated content

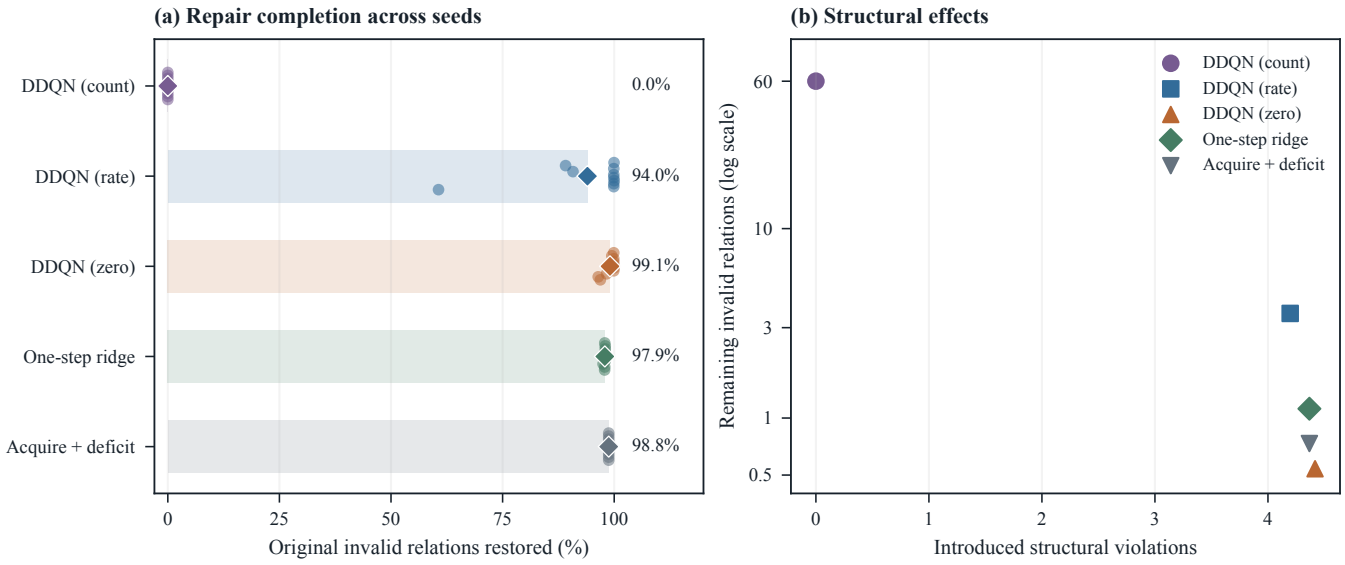


Fig. 4. Policy behavior on thirty new corruptions. (a) Original invalid edge copies restored and kept. Small dots show ten seed means; diamonds show their average. (b) New structural errors and remaining invalid relations. Each seed mean uses thirty scenarios. Panel (b) averages seeds and uses a log axis.

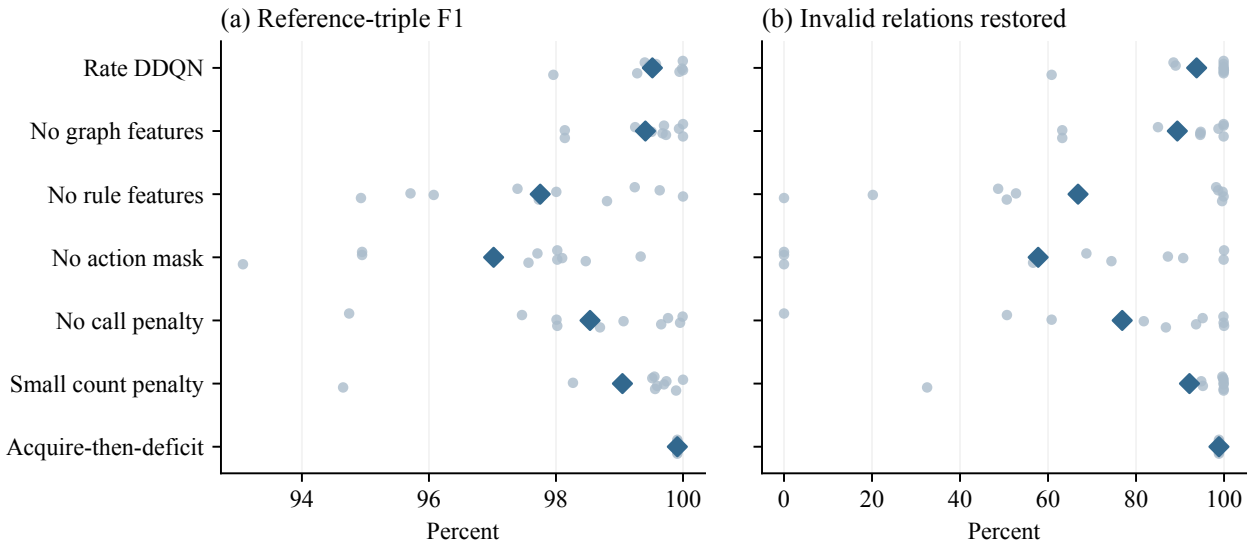


Fig. 5. Rate-reward component tests and small-count control. Dots show ten training-seed means; diamonds show their average. Each mean uses thirty shared scenarios. The heuristic uses a fixed rule.

TABLE III
Rate-reward component tests. Values average ten seed means, each from thirty shared scenarios. Calls count simulated acquisition units. API requests are zero.

Setting	F1	Restoration	Calls	Unavailable
Rate DDQN	0.9952	0.9377	4.45	0.00
No graph features	0.9941	0.8938	4.10	0.00
No rule features	0.9775	0.6682	7.39	0.00
No action mask	0.9702	0.5776	6.23	8.26
No call penalty	0.9854	0.7686	5.54	0.00
Small count penalty	0.9905	0.9214	4.89	0.00
Acquire-then-deficit	0.9991	0.9884	4.00	0.00

TABLE IV
Heuristic time and memory, three runs per size. Time includes construction and actions with `tracemalloc`. Memory counts peak traced Python allocations during these steps. The clean graph and native tensor storage are excluded.

Clean edges	Copies	Time (s)	Memory (MiB)
1,083	1	0.881	1.32
2,166	2	1.839	2.89
4,332	4	4.126	5.46
8,664	8	8.805	11.35

at larger graph sizes.

On the same heuristic action logs, the small-count burden grows by 7.06 times from one to eight copies. The rate burden changes by 0.88 times. These scores measure penalty scaling with matched actions.

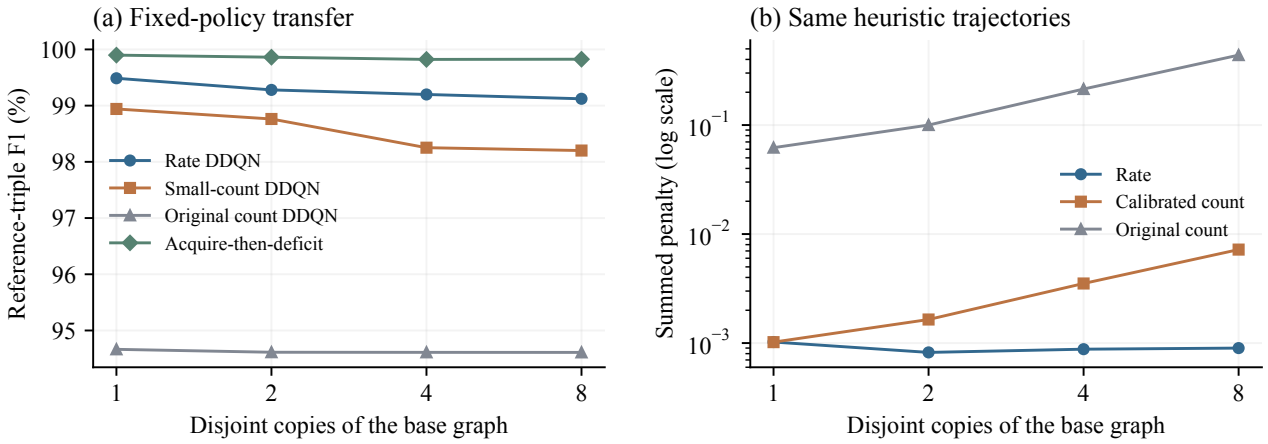


Fig. 6. Graph-size test with disjoint copies. (a) Mean F1 across ten model seeds and five scenarios per size. (b) Three penalties on the same heuristic action logs. All models train at the base size.

TABLE V
Equal-call candidate counts, averaged over thirty document shuffles. Declarations give entity and relation types. Constraints give patterns or statements.

Calls	Strategy	Docs	Decl.	Constraints
100	Deletion	100	468	702
100	Augmentation	100	501	857
100	Dual	50	512	795
1000	Deletion	1000	2634	6044
1000	Augmentation	1000	2887	7787
1000	Dual	500	2970	7126
4000	Deletion	4000	7249	21299
4000	Augmentation	4000	7627	28407
4000	Dual	2000	8081	25901

C. Candidates from Two Prompts

1) Shared Documents and Equal Calls: The paired archive has 32,955 distinct deletion candidates and 41,683 augmentation candidates. Their union has 71,656. These totals include entity and relation declarations. The union is 71.9% larger than augmentation alone. It uses 9,456 calls, compared with 4,728.

Equal-call tests use budgets of 100, 500, 1,000, 2,000, and 4,000. We use thirty seeded shuffles of the paired document list. One strategy processes B documents once each. Dual generation processes the first $B/2$ documents with both prompts. The call count is equal; the document count differs. The saved code removes identical candidates and counts declarations apart from constraints. Table V gives selected budgets. All budgets and sample lists are released.

Augmentation yields more distinct constraints at every equal-call budget. At 1,000 calls, its mean is 7,787, compared with 7,126 for dual generation. The two prompts add different candidates on the same documents. The next stage tests which candidates lead to repairs.

D. Executing Generated Rules

We test candidate execution, repairs from each strategy, and the effect of acquisition order.

TABLE VI
Direct execution of saved type patterns on RuleTest-94: 64 defective and 30 clean cases.

Strategy	Patterns	TP	FP	Recall	F1
Deletion	5794	10	0	0.156	0.270
Augmentation	12617	0	0	0.000	0.000
Dual	18143	10	0	0.156	0.270

1) Direct Execution of Saved Patterns: The archive contains 181,851 candidate copies. Exact allowed and forbidden type patterns compile to 18,143 distinct rules. Each retains its document, strategy, response line, category, and position. Two distinct patterns match RuleTest-94. Of 32,360 compilable copies, 32,348 use terms absent from the relevant test positions. Nine use test vocabulary but match zero records; three match a record. The archive and suite thus share few applicable patterns.

The union detects 10 of 64 defects and flags zero of 30 clean cases. Recall is 0.156 and F1 is 0.270. Augmentation makes zero positive predictions; its precision is undefined. Allowed patterns grant allowed rules. A conflicting allowed rule and forbidden rule cause the checker to skip the decision. This suite has zero conflicts. Logs keep unsupported and malformed candidates. Each strategy is scored on all 94 cases.

2) Generated Rules in a Repair Loop: The interface in Section III-E activates exact saved type patterns on RuleTest-94. It updates detections and the removal mask, removes allowed targets, and saves the next state. Deletion activates 5,794 patterns, augmentation 12,617, and the union 18,143. Deletion and the union each remove ten defective records and zero clean records. Augmentation changes zero records. All thirty clean records remain. Removed records have no replacements.

Four fixed schedules change acquisition order and repair timing. All give the same final result. Both matching patterns come from deletion: one allowed rule matches ten records and one forbidden rule flags another ten. Augmentation matches zero records.

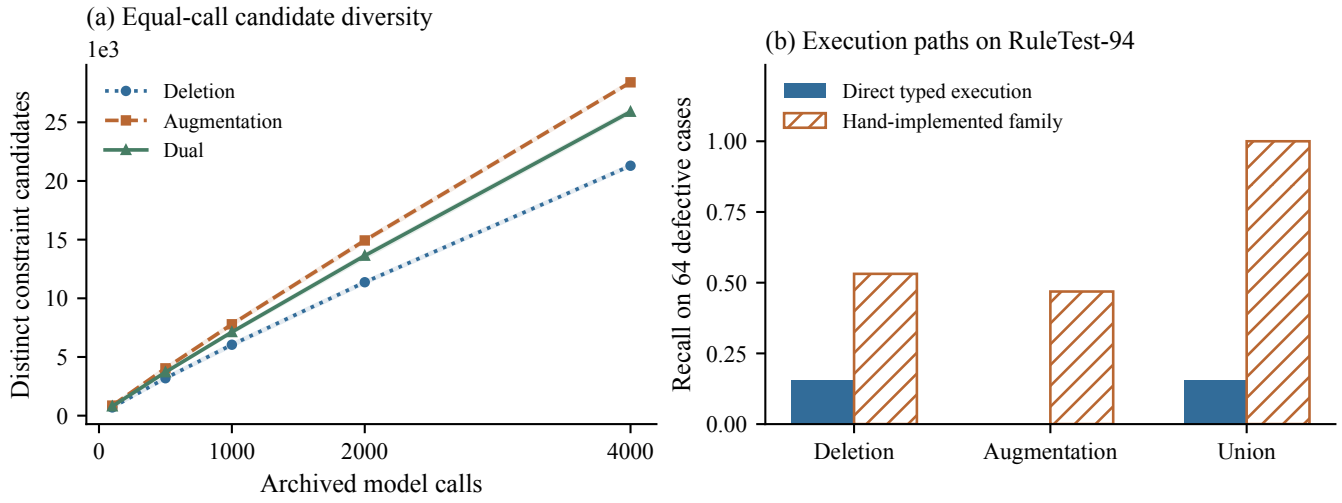


Fig. 7. Rule-generation scores. (a) Distinct constraints at equal call budgets. Shading shows the 2.5–97.5 percentiles across thirty document shuffles. (b) Recall for direct candidate execution and hand-written family checks.

TABLE VII
Saved generated rules on RuleTest-94. Removal deletes records flagged by a forbidden type rule.

Packet	Rules	Defect removals	Clean removals	Retained
Deletion	5794	10	0	84
Augmentation	12617	0	0	94
Union	18143	10	0	84

TABLE VIII
Type-rule coverage on RuleTest-94 and three CSV snapshots. All 18,143 patterns are available. Raw snapshots have zero typed records, so the checker skips them.

Input	Records	Typed	Matching rules	Flagged
RuleTest-94	94	94	2	10
Government	17,939	0	0	0
Finance	8,180	0	0	0
Environment	151	0	0	0

All 26,270 edge copies in the three raw snapshots lack required endpoint types. The checker therefore skips them (Table VIII). Appendix I gives the type-source audit and sixteen schedule traces. The interface loads saved banks and executes fixed schedules.

3) Document Development Tests: The document tests use the four-action environment in Section III-E. The first typed DocRED pilot compiles rules from both prompts and removes zero records. A source-rule version then uses ten development pairs with twenty documents, given entity types, and injected records. It checks quotes against the original source. The reward counts discovered errors apart from edit-created errors. The expansion rule requires source-triggered repair in at least two episodes, higher mean F1, and at least 99% reference facts kept. Appendix C-A gives the protocol and both rounds.

Round 1 raises F1 from 94.42% to 94.59%. It keeps 98.34% of reference facts and loses four. Round 2 uses the same records and reaches 95.69%. It loses zero reference facts and removes seven of 31 injected items. All seven removals use deletion type rules. Zero source-contradiction rules compile. Both rounds end at the expansion check.

All-path replay measures the best available schedules (Table XV). In round 1, the highest proxy return gives 91.63% reference F1, below stop. In round 2, the best allowed reference-scored schedule reaches 95.69%, equal to acquire-then-repair. Order changes discounted return and intermediate quality, while the final F1 remains the same.

A new fixed protocol uses twenty other development documents and an explicit JSON format (Appendix D). Thirty-nine of forty Gemma [29] outputs pass the format checks. Six source-contradiction rules compile. Acquiring both banks then repairing raises F1 from 94.02% to 96.63%. It removes 17 of 28 injected items and loses four reference facts. The mean share of reference facts kept is 98.14%, compared with the 99% threshold. Source contradictions enable three injected-item removals and two reference losses at their pre-edit states. Deletion alone gives the same final graphs with two packets. Both banks use four packets. Augmentation adds zero unique removals. The best allowed safe schedule reaches 96.75% F1, 0.12 points above the strongest fixed schedule. This run also ends at the expansion check.

Admission replay tests the same candidates (Appendix D-A).

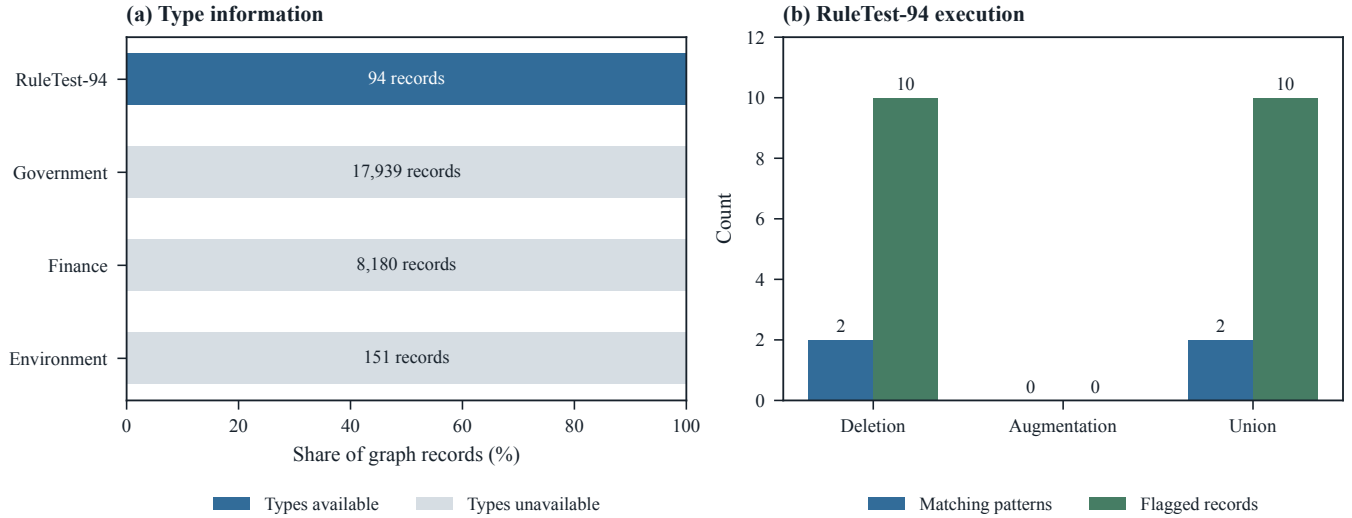


Fig. 8. Rule coverage. (a) Endpoint types in each snapshot; bar labels give record counts. (b) Matching patterns and flagged records in RuleTest-94. The checker skips records with missing types.

Source-link checks keep all 407 rules and reproduce the repairs. The independent-approval registry is empty, and the hard-schema mapping is absent. Both approval settings keep zero rules. F1 stays at 94.02%, with zero losses and zero repairs.

E. Time and Cost

Scheduling uses local graph operations and simulated acquisition units. Its actual API request count is zero. Candidate-count tests use successful saved responses. Appendix A lists their recorded metadata.

The two source-development rounds use 80 Gemma requests for 80 responses. The new follow-up uses forty requests with zero transport failures or retries. It uses 255,391 prompt tokens and 58,346 completion tokens. Mean request time is 13.80 seconds, excluding spacing between requests. Saved-output replay makes zero requests. Packet counts measure replay acquisition. Appendix I-B gives training and extraction times.

F. Discussion

Reward design changes which repairs the policy completes. The rate penalty improves over the original count penalty. The action mask raises final F1. The simple heuristic has the highest mean reference F1. The small-count control shows the role of penalty size.

Useful acquisition needs rules that match records and allow correct repairs. The saved archive has few patterns applicable to RuleTest-94. Raw snapshots also need entity types. With given types, generated rules repair development inputs. In round 2, the best schedule and acquire-then-repair have equal final F1. In the new document pairs, source rules remove injected items and some reference facts. Augmentation adds zero unique removals. These results point to rule review and broader useful coverage as the next steps.

The scheduling score uses controlled errors and trusted restoration records. Document tests score recovery after injected changes. Independent review can assess rule and edit meaning. New domains can test transfer. Appendix H gives structural and extraction scores.

V. Conclusion

We use Double DQN to divide a fixed budget between graph repair and rule acquisition. Deletion and augmentation prompts generate rule candidates. The tests follow three stages: scheduling, generation, and rule execution.

The count penalty charges for temporary duplicate edges and blocks useful relation repair. The rate penalty raises Double-DQN triple F1 from 94.67% to 99.49% on new corruption scenarios. Acquire-then-deficit reaches 99.91%. Removing the action mask lowers F1 by 2.50 points after correction for multiple tests. Small-count DDQN reaches 99.05% F1 at the base size; rate DDQN reaches 99.52%. Their paired difference is -0.47 points (95% CI: $[-1.73, 0.46]$). These tests show how reward size and allowed actions affect repair.

The two prompts produce different candidates on the same documents. Augmentation gives more constraints at equal call budgets. Direct execution removes ten designed defects. Deletion alone covers all ten removals. The second document-development round reaches 95.69% F1, removes seven injected items, and loses zero reference facts. All seven removals use deletion type rules. The best allowed schedule has the same final F1 as acquire-then-repair.

A new test on twenty other development documents activates source rules and reaches 96.63% F1. It removes 17 injected items and loses four reference facts. Deletion alone gives the same final graphs. Requiring approval with the current empty review sources keeps all reference facts and makes zero repairs. The next step is to improve rule review and useful rule coverage, then test learned scheduling against the same simple baselines. Independent rule and edit review can check meaning. New document domains can test transfer.

Appendix A
Generation Prompts and Recorded Settings

The supplement gives the original Chinese prompt templates, SHA-256 hashes, and English translations. The descriptions below explain each task in simple English. Saved records contain parsed responses and removed spans. The original templates remain in the experiment archive.

A. Deletion Completion Prompt

The deletion prompt asks the model to read the original and changed service document. It asks for rule information from the removed phrases and reusable graph constraints. The response must follow the supplied JSON format. Inputs include the service item, original text, changed text, and removed spans. Candidates use general types and relations.

The JSON fields are strategy, service_item, analysis, removed_fragments, missing_rule_candidates, and proposed_rules. The last field contains the seven categories below. The runner makes one call per document and strategy.

B. Augmentation Expansion Prompt

The augmentation prompt asks for up to three reasonable extra sentences for the service document. It then asks for reusable graph rules from these additions. The response must follow the supplied JSON format. Inputs include the service item and original text.

Its fields are strategy, service_item, analysis, added_clauses, and proposed_rules. Both prompts share seven categories: entity types, relation types, forbidden patterns, allowed patterns, hierarchy statements, geographical hierarchy patterns, and statements about required steps.

C. Defaults and Recorded Settings

The script defaults to Qwen3-32B and temperature 0.2. It removes up to five spans of at most six characters and requests up to three extra sentences. The client leaves the completion limit and top-p unset. Historical records store document ID, service item, strategy, parsed result, and removed spans when present. Full per-call settings and rendered requests are absent from these logs. The separate 45-document extraction test records Qwen3.8-27B-no-thinking.

Appendix B
RuleTest-94 and Direct Execution

RuleTest-94 has 30 clean and 64 defective records: 10 hierarchy reversals, 10 absurd relations, 10 reverse-supervision cases, 30 required-step or missing-field cases, and four specialist cases. The scorer reads expected_detection. A pass label is negative; fail and specialist_miss are positive. The suite provides these designed labels.

The union of hand-written family checks detects all 64 defects and flags zero clean records. Its precision Wilson interval [30] is $[0.943, 1.000]$; specificity is $[0.886, 1.000]$. Section IV-D1 gives direct execution of saved candidates.

Each compiled rule has an ID, allowed or forbidden kind, exact subject-type/relation/object-type pattern, and original source positions. Exact pattern matches decide which records it covers. Permissions allow a combination. Prohibitions flag it. A conflict causes a skipped decision. The log retains unsupported natural-language candidates.

Appendix C
Document Pilot with Given Types

We split source-grouped DocRED [27] documents into 180 training, 60 development, and 60 test items from the original annotated training set. Entity IDs and mention types are given. Public source inputs and reference relations are stored apart.

Ten fixed development documents each receive two rule responses and one extraction response from Gemma. The 30 outputs use 34 requests including retries. All twenty rule responses parse. They give 263 compiled candidates: 156 allowed and 107 forbidden patterns. One extraction fails the fixed parser and becomes an empty graph. The other outputs give 106 executable relations. Five two-document episodes run stop and four acquisition/repair schedules. All schedules remove zero records and reach the same final graph within each episode. The generated forbidden rules thus match zero repair targets in these outputs. Records and code are in exps/paper2_docred.

A. Source-Rule Development Test

This version keeps the two prompts and four-action scheduler and adds source rules for each document. A candidate marks a record as supported, contradicted, or insufficient. The compiler checks an exact quote in an original numbered sentence and saves character positions. Augmentation sentences serve as generation hints. Insufficient evidence or conflicting support and contradiction cause a skipped decision. An allowed type rule leaves a source contradiction active.

Let $V(g; R)$ be the set of nonconflicting violations. Let N be the initial record count, using one for an empty input. We use $G(g; R) = 1 - |V(g; R)|/N$ and coverage C over initial records. The graph-change term checks both graphs under updated rules:

$$\begin{aligned} r_t = {} & 0.5\,[G(g_{t+1}; R_{t+1}) - G(g_t; R_{t+1})] \\ & + 0.5(C_{t+1} - C_t) - 0.004a_t \\ & - 0.00002d_t - |V_{\text{edit},t}|/N, \end{aligned} \tag{15}$$

Here a_t counts acquired responses and d_t counts removed records. $V_{\text{edit},t} = V(g_{t+1}; R_{t+1}) \setminus V(g_t; R_{t+1})$ counts edit-created violations. Acquisition can reveal an existing violation with zero charge for creating it. A two-record synthetic test changes discounted acquire–repair return from -0.266519 to 0.483481 at discount 0.95. This test checks reward accounting. The model judges source support; the compiler checks the quote text.

Before calls, we fix the first ten development pairs, or 20 documents. Each pair has one reference graph and one reference graph with added endpoint or relation changes. The added count is $\lceil 0.2|G^*| \rceil$. References supply input construction and scoring. Prompts and policies use public inputs. Each

TABLE IX
Source-rule test on ten fixed document pairs. F1 and facts kept are episode means. Lost counts removed reference facts. Schedules are fixed.

Schedule	F1 (%)	Facts kept (%)	Lost
Stop	94.42	100.00	0
Acquire then repair	94.59	98.34	4
Repair when feasible	91.63	93.51	18
Uniform feasible	94.42	100.00	0

observation has 14 summaries and an action mask. An episode allows four responses, two per strategy, and ten steps. The removal guard blocks actions that would empty a nonempty graph.

The 40 outputs use 40 Gemma requests. They compile to 357 type rules, 61 source-support rules, and 6 source-contradiction rules. Source rules trigger removal in 4 of ten episodes. The fixed expansion check requires at least two such episodes, higher mean reference F1, and at least 99% reference facts kept. This version ends at that check. Scores measure reference recovery and injected-item removal. Saved-output replay uses zero API calls.

a) Second development round.: Round 2 keeps the same twenty documents, compiler, reward, budget, and expansion check. Prompts add readable entity names and explain property direction, historical roles, and absent versus conflicting evidence. Forty more requests complete with zero transport failures. Thirty-eight responses pass the fixed parser. They yield 243 type rules, 14 source-support rules, and zero source-contradiction rules.

Acquire-then-repair and repair-when-feasible both reach 95.69% reference F1, compared with 94.42% for stop. Each removes seven of 31 injected items and loses zero reference facts. All seven removals use type rules. The source-repair check ends the two-round study.

The compiler rejects 120 type candidates in round 1 and 257 in round 2 that copy $\mathrm{allowed\ or\ forbidden}$ instead of choosing one kind. Both rounds use the fixed parser. The logs retain round 1 reference losses and round 2 source-rule results.

Appendix D
New Rule-Execution Test

A new fixed protocol uses twenty other development documents. We take the first ten pairs in development order whose members are absent from prior pilot lists and response logs. Each pair has a reference graph and a graph with injected records. There are 226 reference facts and 28 injected items. The test scores controlled reference recovery.

The prompts give exact JSON choice examples and allow at most twenty rules and three extra sentences. Duplicate keys, invalid choices, or long lists invalidate the whole response. Candidates then pass the fixed vocabulary and quote checks. Forty requests use $\mathrm{google/gemma\text{-}4\text{-}26B\text{-}A4B\text{-}it}$, temperature zero, and a 4,000-token limit. Transport failures and retries are zero. One response exceeds the rule limit and becomes an empty packet.

The 39 valid responses give 109 allowed and 36 forbidden type rules, 256 source-support rules, and six source-contradiction rules. All six contradictions come from deletion.

TABLE X
New rule test on ten document pairs. F1, facts kept (Kept), and packet counts are pair means. Removed injected items (Rem.) and lost reference facts (Lost) are totals. Both-bank schedules give the same final graphs.

Schedule	F1 (%)	Kept (%)	Rem.	Lost	Packets
Stop	94.02	100.00	0	0	0.0
Both banks	96.63	98.14	17	4	4.0
Deletion only	96.63	98.14	17	4	2.0
Augmentation only	94.24	100.00	1	0	2.0
Uniform feasible	95.58	100.00	8	0	2.1

The compiler skips 163 insufficient-evidence decisions and rejects nineteen candidates with quotes absent from the original text.

The eight fixed schedules are stop, both acquisition orders with immediate or deferred repair, deletion only, augmentation only, and uniform allowed actions including stop. Eighty replays agree with all 384 terminal paths. Random scheduling uses one fixed seed per pair. Acquire-then-repair leads stop by 2.61 F1 points. The descriptive 95% paired-bootstrap interval is $[0.81, 4.30]$ from 10,000 draws of ten pairs. The mean share of reference facts kept is 98.14%, with four lost facts; the threshold is 99%.

We remove source-contradiction rules at each pre-edit state to check which targets lose their flags. This finds three injected-item removals across two episodes and two reference losses that need source contradictions. Deletion has sixteen unique injected-item removals compared with augmentation. Augmentation has zero.

The protocol also checks the best allowed safe schedule, or oracle. It requires at least 0.5 F1 points over the strongest fixed schedule, or at least one packet of mean savings at equal quality. Savings must occur in two or more episodes. A safe path keeps at least 99% of reference facts in its episode. The oracle uses scorer references after replay. Its mean F1 is 96.75%, 0.12 points above deletion alone. One episode has zero safe paths matching the baseline F1. The format check passes; the other expansion checks stop this run. This protocol changes both prompts and documents. Inputs, rules, traces, and costs are in $\mathrm{exps/paper2_rule_feasibility_20261004/}$.

a) Checking lost reference facts.: Recompiling forty responses and replaying eighty schedules reproduces the saved outputs. Two reference losses come from forbidden type rules: $\mathrm{LOC} \times P150 \times \mathrm{LOC}$ and $\mathrm{LOC} \times P37 \times \mathrm{MISC}$. Two more come from source-contradiction rules for university–country relations. Both cite the same biography sentence. It mentions both entities and leaves the country relation unstated. The biography used for the official-language reference also leaves the country's official language unstated.

Further family-filter tests use the same banks. Under acquire-then-repair, removing source contradictions keeps fourteen injected-item removals and two reference losses, at 96.57% F1. Removing forbidden type rules keeps four removals and two losses, at 94.33% F1. Removing both gives zero losses and zero repairs, at 94.02% F1. These filters use public rule families. The full audit has 320 replays including the original schedules. The results point to reviewing rules

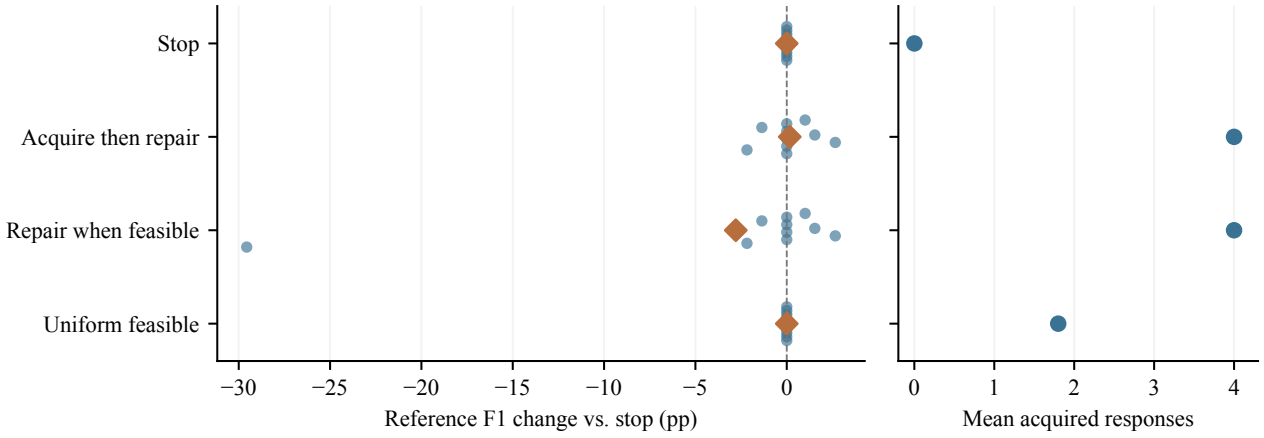


Fig. 9. Source-rule replay. Dots show ten episode F1 changes from stop; diamonds show their mean. The right panel counts acquired saved responses.

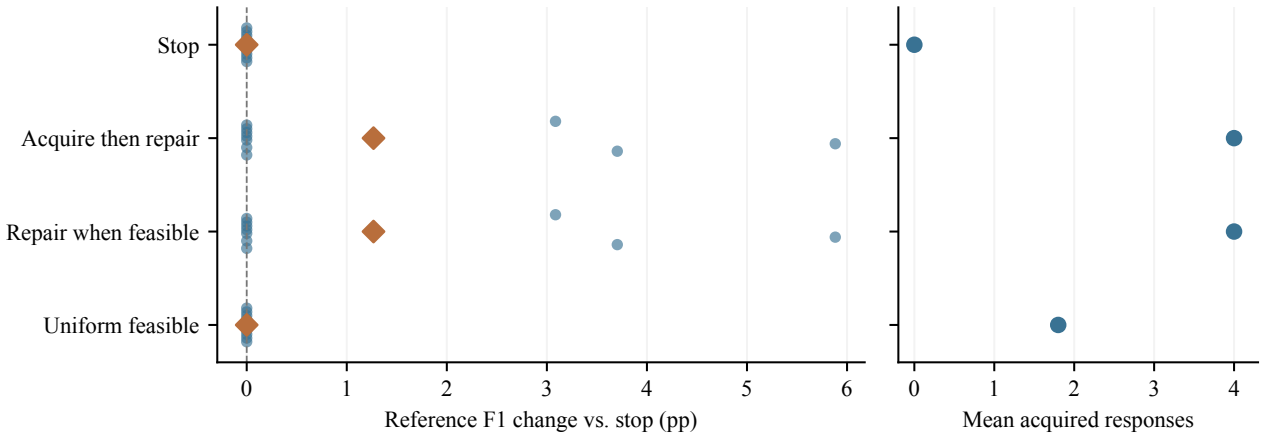


Fig. 10. Round 2 on the same ten development pairs. Both full schedules reach 95.69% mean F1 and keep every reference fact. Seven removals use type rules.

TABLE XI
Rule approval on the same development inputs with acquire-then-repair. Removed counts injected items; lost counts reference facts.

Admission	Rules	F1 (%)	Removed	Lost
Provenance only	407	96.63	17	4
Independent approval (empty)	0	94.02	0	0
Hard schema (no mapping)	0	94.02	0	0

before activation.

A. Rule Approval and Coverage

Three admission settings replay the same ten document pairs under eight fixed schedules, giving 240 replays. The first checks source links. The second requires independent approval from an empty registry. The third requires a hard-schema mapping from DocRED's coarse types; this mapping is absent. All 407 candidates pass source-link checks. The two approval settings keep zero rules, including allowed rules and support rules.

The eighty source-link traces match saved actions, edits, and rewards. All-path replay gives 384 terminal paths with source-link checks and 190 for each empty approval setting. Requiring approval keeps all four reference facts lost in the original run. It also removes all seventeen injected-item repairs. The automatic-review protocol is fixed; its model comparison remains a later step. Inputs, decisions, and costs are in exps/paper2_rule_verification_20261008/.

Appendix E
Fixed Reward-Test Protocol

Training uses seeds 20000–20009. For seed s and episode $e \in \{0, \ldots, 249\}$, the environment seed is $100000s + e$. Separate ridge rollouts use seeds 30000–30009 and the same episode formula. Development scenarios are 920000–920019. Test scenarios are 930000–930029. They are disjoint from the ten earlier action-test scenarios. All use the same 450-document base graph.

The training manifest fixes sources, graph data, penalties, four neural settings, ridge setup, and statistical comparisons. After forty neural models, ten ridge fits, and development checks, a second manifest records model and scorer hashes. The test evaluates all eight policies on the same thirty scenarios. Ten run-seed means supply paired statistical units.

Policies receive public observations and the current mask. References supply scoring, and the environment holds restoration records. Saved initial graphs and node/edge changes rebuild every action log. Reconstruction checks final scores, remaining errors, masks, reward parts, costs, and returns under all three penalties.

A. Training and Replay Records

The reward study fixes sources, costs, seeds, and episode budgets before training forty neural models and ten ridge baselines. Development checks cover 1,600 outcomes. A second manifest records all fifty models and scorer hashes before the test runs. Thirty test scenarios give 2,400 outcomes. Every predefined policy is retained. The archive saves initial graphs, each graph change, masks, rewards, and reference scores. Earlier versions keep their own checkpoints and results. Training and replay use zero API requests. The earlier extraction test uses Qwen3.8-27B-no-thinking. Appendix A gives historical rule-generation settings.

Appendix F
Earlier Count-Penalty Component Study

Input order is four graph scores; rule precision, recall, and coverage; four scaled error counts; budget; and active shares of high- and low-precision modules. Action order is isolated-node cleanup, duplicate removal, relation repair, missing-endpoint cleanup, deletion acquisition, augmentation acquisition, mixed acquisition, and pruning.

This study has sixty trained models and 110 evaluations on scenarios 900000–900009. DQN, Double DQN, and four component settings use training seeds 10000–10009, 250 episodes, shared network and replay settings, and 260-episode exploration decay. Removing graph features zeros indices 0–3 and 7–10. Removing rule features zeros indices 4–6 and 12–13. Indices start at zero. Other inputs and the mask stay available. The mask also reports which rules allow an action.

The no-mask setting removes action-selection and Bellman masks. The no-call-penalty setting removes training acquisition cost. All evaluations use the count penalty, shared costs, and 18 steps. For AUC, early terminal quality is carried forward to step 18. The archive retains the older short-action log AUC separately.

Two-sided paired sign randomization uses all 2^{10} sign assignments. Intervals use 10,000 whole-scenario bootstrap draws with seed 42. Holm correction covers ten comparisons: full Double DQN against the heuristic and four component settings, for final quality and AUC. These tests reuse the ten original scenarios.

A. Earlier Policy Results

Count-penalty Double DQN reaches final quality 0.96855 and AUC 0.94514, with 6.2 acquisition units. Acquire-then-deficit reaches 0.98248 and 0.95797 with four units. Double DQN minus the heuristic is -0.01392 for quality and -0.01283 for AUC. Both Holm-adjusted p values are 0.0195 (Table XIV). DQN reaches quality 0.96920.

Double DQN adds zero violations in ten runs. The heuristic adds 4.8 per run. For $\mathcal{R} = \sum_{t=0}^{T-1} 0.95^t r_t$, their mean returns are 0.3660 and 0.3508. Section IV-B shows that Double DQN avoids relation repair. These returns use the count penalty. Logs retain each reward part and new error ID.

B. Retrained Component Tests

Ten models per setting train for 250 episodes. Four settings remove graph features, rule features, the mask, or training acquisition cost. Feature removal applies in training and evaluation. These settings retain the mask. The no-mask setting changes action selection and Bellman targets. The no-call-cost setting changes the training reward. All evaluations use the same corrected count reward and costs.

Removing the mask lowers final quality to 0.94800 and gives 10.7 invalid actions per run. Its quality test has Holm $p = 0.0195$; its AUC test has $p = 0.8340$. Graph-feature, rule-feature, and call-cost tests have adjusted $p > 0.05$ across ten comparisons. The mask improves final quality.

Exact paired tests use all 2^{10} sign assignments. Holm correction covers full Double DQN against the heuristic and four component settings for both scores. Intervals use 10,000 paired bootstrap draws. Sixty models complete 15,000 training episodes. Earlier checkpoints keep their original environment version.

Appendix G
Code and Result Files

Original checkpoints, graph steps, and generation records remain in their experiment directories. $\mathrm{exps/paper2_offline_revision/}$ contains source hashes, forty checkpoints, training histories, paired action logs, equal-call samples, rule sources, label checks, and publication scripts. It includes manuscript-to-result checks and offline replay commands.

$\mathrm{exps/math_revision_20260925/}$ contains the corrected environment, sixty checkpoints, full action logs, the shared-profile Paper1 revision, and the rule interface. It includes a source manifest and replay command.

$\mathrm{exps/paper2_reward_validation/}$ contains training and test manifests, forty neural checkpoints, ten ridge models, exploration data, action logs, paired tests, reconstruction checks, and figure/table scripts. Its README gives local commands with zero API requests or model downloads.

$\mathrm{exps/paper2_rule_integration/}$ fixes the raw CSV snapshots, keeps matching and nonmatching rules, and saves sixteen schedule traces. Runtime inputs and scorer labels are stored apart. Synthetic interface tests have their own counts.

a) Checking all scheduling paths.: $\mathrm{exps/papers_readiness_20261002/}$ enumerates every allowed acquisition, repair, and stop path under four packets and ten steps. It uses the two development rounds and their twenty shared documents. References score paths after enumeration. Table XV gives two bounds. The schedule oracle uses allowed action paths. The relaxed bound can remove each injected item reached by a stored proposal.

TABLE XII
Policy scores on ten paired scenarios: mean and sample standard deviation. Calls count simulated acquisition units.

Policy	Final quality	AUC_{18}	Calls	Invalid	New errors	Return
Random valid	0.9642 ± 0.0065	0.9254 ± 0.0152	10.2	0.0	2.9	0.3163
Alternating valid	0.9677 ± 0.0007	0.9292 ± 0.0020	10.0	0.0	4.0	0.3102
Rule-first valid	0.9747 ± 0.0004	0.9365 ± 0.0024	8.0	0.0	4.6	0.3255
Acquire + deficit	0.9825 ± 0.0001	0.9580 ± 0.0014	4.0	0.0	4.8	0.3508
DQN	0.9692 ± 0.0037	0.9454 ± 0.0079	5.2	0.0	0.0	0.3688
Double DQN	0.9686 ± 0.0034	0.9451 ± 0.0096	6.2	0.0	0.0	0.3660
Model-informed lookahead	0.9711 ± 0.0048	0.9516 ± 0.0027	5.1	0.0	0.2	0.3750

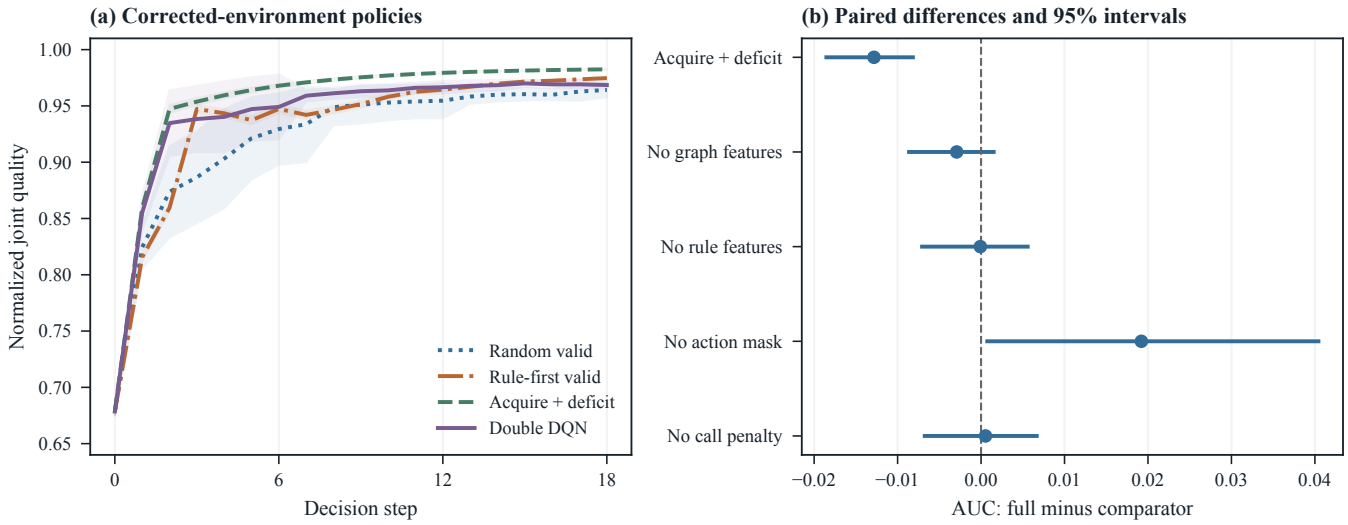


Fig. 11. Earlier count-penalty study on ten reused scenarios. (a) Mean quality with one-standard-deviation bands. Final scores carry forward to step 18. (b) Full Double DQN minus each baseline in AUC, with 95% paired-bootstrap intervals. Table XIV gives adjusted tests.

TABLE XIII
Component tests in the corrected environment. Each setting has ten retrained models.

Setting	Final quality	AUC_{18}	Calls	Invalid	New errors	Return
Double DQN	0.9686 ± 0.0034	0.9451 ± 0.0096	6.2	0.0	0.0	0.3660
No graph features	0.9713 ± 0.0041	0.9481 ± 0.0028	5.1	0.0	0.9	0.3693
No rule features	0.9692 ± 0.0062	0.9452 ± 0.0034	7.6	0.0	0.5	0.3594
No action mask	0.9480 ± 0.0215	0.9259 ± 0.0278	4.9	10.7	0.0	0.3430
No call penalty	0.9659 ± 0.0051	0.9446 ± 0.0044	8.1	0.0	0.0	0.3596

In round 1, the schedule oracle reaches 95.45% F1. Paths with the highest proxy reward reach 91.63%, compared with 94.42% for stop. Wrong rules thus make the proxy reward favour lower reference F1. In round 2, both oracle bounds reach 95.69%, equal to acquire-then-repair. Deletion type rules cover all seven removable injected items. Augmentation and source rules add zero removals. All-packet repair reaches identical final graphs across acquisition orders. Discounted returns vary with order. This further development analysis has 598 terminal paths.

The JSON-contract draft gives legal choices, rule families, candidate limits, and empty outputs. The unchanged compiler reproduces rules and rejections for all 80 old responses. Appendix D tests this format on other documents. The old rounds retain their original parser and scores.

Appendix H
Further Repair Tests

These tests measure detector coverage, structural repair, and extraction. Section IV-B gives policy comparisons. Section IV-D gives generated-rule effects.

A. Separate Rule-Family Check

Hand-written deletion-family checks cover 34 required-step, missing-field, and specialist defects. Augmentation-family checks cover 30 structural defects. Their union detects all 64 and flags zero clean records under the stored labels. The functions directly encode these defect families. Table VI tests saved generated patterns.

B. Constraint-Checking Baseline

The SHACL-style RDF baseline receives unique triples and checks selected schema shapes. Its inputs contain graph

TABLE XIV
Full Double DQN minus each baseline. Each interval is a paired-bootstrap 95% CI. Holm correction covers ten tests.

Baseline	Metric	Difference	95% interval	p_{Holm}
Acquire + deficit	Final	-0.01392	[-0.01604, -0.01205]	0.0195
Acquire + deficit	AUC	-0.01283	[-0.01853, -0.00817]	0.0195
No graph features	Final	-0.00273	[-0.00601, -0.00007]	0.8340
No graph features	AUC	-0.00292	[-0.00862, +0.00151]	1.0000
No rule features	Final	-0.00068	[-0.00406, +0.00268]	1.0000
No rule features	AUC	-0.00008	[-0.00706, +0.00558]	1.0000
No action mask	Final	+0.02055	[+0.00897, +0.03502]	0.0195
No action mask	AUC	+0.01921	[+0.00075, +0.04042]	0.8340
No call penalty	Final	+0.00269	[-0.00031, +0.00572]	0.8340
No call penalty	AUC	+0.00053	[-0.00674, +0.00667]	1.0000

TABLE XV
All scheduling paths on the same ten development pairs per round. F1 averages pairs. Oracle scores use reference labels after replay.

Diagnostic	Round 1	Round 2
Feasible terminal paths	363	235
Stop F1 (%)	94.42	94.42
Executable oracle F1 (%)	95.45	95.69
Selective-removal upper F1 (%)	95.60	95.69
Reward-optimal F1 (%)	91.63	95.69
Oracle correct facts lost	0	0
Oracle injected items removed	5	7

TABLE XVI
Hand-written family checks scored against stored suite labels. Table VI gives compiled generated patterns.

Detector	TP	FP	FN	Recall	F1
Deletion-family	34	0	30	0.531	0.694
Augmentation-family	30	0	34	0.469	0.638
Family union	64	0	0	1.000	1.000

records and shapes. This test measures the shapes' coverage.

After RDF duplicate removal, all inputs pass the selected shapes. The baseline removes zero further triples. Its mean quality score is 71.85 under the separate metric. The selected shapes check schema form. The quality metric also includes source and required-step checks.

C. Deterministic Structural Repair

A local TNEWS/CLUE test builds a document–category–entity KG from 3,000 balanced news titles across 15 categories. A fixed extractor builds the graph. We add duplicate triples, invalid relations, and isolated nodes, then run the repair pipeline. This test uses local code and zero model or API calls.

Repair removes injected invalid relations, duplicate triples, and isolated noise nodes. Structural Q rises from 94.93 to 100.00. The final graph has fewer nodes and relations than the clean extraction. exps/external_benchmark_runner.py generates the report.

D. Model Extraction Test

The model-extraction test uses 45 TNEWS/CLUE titles across 15 categories and Qwen3.8-27B-no-thinking. Category

TABLE XVII
SHACL-style structural checks on three domains.

Domain	Raw Triples	RDF Dedup.	Removed	Q
Gov. Affairs	13,892	10,771	0	79.87
Finance	2,264	539	0	70.42
Env.	2,345	378	0	65.25
Average	6,167	3,896	0	71.85

TABLE XVIII
Local TNEWS/CLUE graph-quality test.

Stage	Nodes	Relations	Invalid	Q
Clean extraction	10,871	20,827	0.000	99.89
Corrupted KG	11,414	22,909	0.050	94.93
Repaired KG	10,822	19,799	0.000	100.00

labels supply reference values for category inference. Given title keywords supply weak mention labels. We score parsing, category accuracy, keyword recall, and structural filtering.

All 45 responses parse. The calls take 1,190.90 seconds. Every output follows the allowed schema. The filter changes zero triples, and both input and output structural quality are 100.00. Category accuracy is 0.556 and keyword recall is 0.178. These scores point to category choice and missing mentions as targets for better extraction.

Appendix I
Rule Coverage and Time Records

A. Types and Fixed-Schedule Traces

The interface in Section III-E loads deletion or augmentation rules, removes unopposed forbidden matches, or stops. Two acquisition orders with immediate or deferred repair give four schedules per input. RuleTest-94 and three snapshots give sixteen traces. All four schedules remove the same ten defects and zero clean records on RuleTest-94. Late allowed rules and conflicts are zero. Separate synthetic tests check conflicts and late-allowed rule logging.

Government, finance, and environment snapshots have 17,939, 8,180, and 151 edge copies. All node types are Unknown. The 26,270 edges therefore lack required endpoint types. The source audit checks 51 node CSV files, three document JSONL files, and two extraction checkpoints. It finds zero traceable types. Enhanced exports use Enhanced as

TABLE XIX
Model extraction on TNEWS/CLUE titles.

Metric	Raw LLM KG	Repaired KG
Parse success	1.000	—
Category accuracy	0.556	0.556
Keyword recall	0.178	0.178
Documents with triples	1.000	1.000
Avg. triples / doc	3.58	3.58
Invalid triple rate	0.000	0.000
Duplicate triple rate	0.000	0.000
KG quality score	100.00	100.00

a processing tag. Export identity requires a traceable mapping. The runtime skips these records. Table VIII separates missing types from zero pattern matches. Appendix H-B uses its own stored inputs.

Both RuleTest-94 matches come from deletion. One allowed rule matches ten records; one forbidden rule flags ten more. Augmentation matches zero. All schedules give the same final records. Further scheduling tests need trusted types and broader useful rule coverage. Loading one strategy bank reads 4,728 saved responses and uses zero API requests.

B. Time Details

The original twenty-model run records 1,278.99 seconds of CPU time and zero API calls. The sixty-model component study and forty-model reward study also use zero requests. Ridge adds ten fits from 2,500 random episodes. Rate-reward component tests add fifty models with zero requests. The archive gives training and evaluation times. Evaluation includes policy inference and graph steps. Construction time is excluded. The model-informed baseline also times copied-state trials.

The 45-document extraction test uses 45 calls and 1,190.90 seconds. The rule archive has 9,456 successful responses. Complete historical latency, token, retry, and failure records are absent. Equal-call tests count candidates per saved successful response. Scheduling uses simulated acquisition units.

References

[1] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2022.

[2] H. Paulheim, "Knowledge graph refinement: A survey of approaches and evaluation methods," *Semantic Web*, vol. 8, no. 3, pp. 489–508, 2017.

[3] A. Zaveri, A. Rula, A. Maurino, R. Pietrobon, J. Lehmann, and S. Auer, "Quality assessment for linked data: A survey," *Semantic Web*, vol. 7, no. 1, pp. 63–93, 2016.

[4] D. Kontokostas, P. Westphal, S. Auer, S. Hellmann, J. Lehmann, R. Cornelissen, and A. Zaveri, "Test-driven evaluation of linked data quality," in *Proceedings of the 23rd International Conference on World Wide Web*, 2014, pp. 747–758.

[5] H. Paulheim and C. Bizer, "Improving the quality of linked data using statistical distributions," *International Journal on Semantic Web and Information Systems*, vol. 10, no. 2, pp. 63–86, 2014.

[6] R. Das, S. Dhuliawala, M. Zaheer, L. Vilnis, I. Durugkar, A. Krishnamurthy, A. Smola, and A. McCallum, "Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://arxiv.org/abs/1711.05851

[7] W. Xiong, T. Hoang, and W. Y. Wang, "DeepPath: A reinforcement learning method for knowledge graph reasoning," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 564–573. [Online]. Available: https://aclanthology.org/D17-1060/

[8] X. V. Lin, R. Socher, and C. Xiong, "Multi-hop knowledge graph reasoning with reward shaping," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 3243–3253. [Online]. Available: https://aclanthology.org/D18-1362/

[9] L. Yao, C. Mao, and Y. Luo, "KG-BERT: BERT for knowledge graph completion," arXiv preprint arXiv:1909.03193, 2019. [Online]. Available: https://arxiv.org/abs/1909.03193

[10] X. Wang, T. Gao, Z. Zhu, Z. Zhang, Z. Liu, J. Li, and J. Tang, "KEPLER: A unified model for knowledge embedding and pre-trained language representation," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 176–194, 2021. [Online]. Available: https://aclanthology.org/2021.tacl-1.11/

[11] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[12] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[14] H. Knublauch and D. Kontokostas, "Shapes constraint language (SHACL)," W3C Recommendation, 2017. [Online]. Available: https://www.w3.org/TR/2017/REC-shacl-20170720/

[15] E. Prud'hommeaux, J. E. Labra Gayo, and H. Solbrig, "Shape expressions: An RDF validation and transformation language," in *Proceedings of the 10th International Conference on Semantic Systems*, 2014, pp. 32–40.

[16] L. A. Galárraga, C. Teflioudi, K. Hose, and F. M. Suchanek, "AMIE: Association rule mining under incomplete evidence in ontological knowledge bases," in *Proceedings of the 22nd International Conference on World Wide Web*, 2013, pp. 413–422.

[17] S. Ortona, V. V. Meduri, and P. Papotti, "RuDiK: Rule discovery in knowledge bases," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1946–1949, 2018. [Online]. Available: https://www.vldb.org/pvldb/vol11/p1946-ortona.pdf

[18] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/0e55666a4ad822e0e34299df3591d979-Abstract.html

[19] M. Qu and J. Tang, "Probabilistic logic neural networks for reasoning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/hash/13e5ebb0fa112fe1b31a1067962d74a7-Abstract.html

[20] M. Qu, J. Chen, L.-P. Xhonneux, Y. Bengio, and J. Tang, "RNNLogic: Learning logic rules for reasoning on knowledge graphs," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://arxiv.org/abs/2010.04029

[21] T.-W. Lin, G. Fierro, H. Li, T. Hong, P. Nuzzo, and A. Sangiovanni-Vinentelli, "Systematic evaluation of knowledge graph repair with large language models," arXiv preprint arXiv:2507.22419, 2025. [Online]. Available: https://arxiv.org/abs/2507.22419

[22] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[23] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 2016, pp. 2094–2100.

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[25] Qwen Team, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[26] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan, "CLUE: A Chinese language understanding evaluation benchmark," in *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, 2020, pp. 4762–4772. [Online]. Available: https://aclanthology.org/2020.coling-main.419/

[27] Y. Yao, D. Ye, P. Li, X. Han, Y. Lin, Z. Liu, Z. Liu, L. Huang, J. Zhou, and M. Sun, "DocRED: A large-scale document-level relation extraction dataset," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 764–777. [Online]. Available: https://aclanthology.org/P19-1074/

[28] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

[29] Gemma Team, "Gemma 4 technical report," arXiv preprint arXiv:2607.02770, 2026. [Online]. Available: https://arxiv.org/abs/2607.02770

[30] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.